



جامعة اللاذقية
كلية الهندسة المعلوماتية
مقرر برمجة التطبيقات الشبكية

عنوان المشروع:

Task Server Pro: Centralized Task Management System

إشراف:

م. راما الخير

أسماء الطلاب:

حسن أكرم يوسف
أناغيم سلطان محمد

بشرى يعقوب ديوب
معالي محمد محرز

نوع المسار:

Client/server

المشروع المرآة:

P2P Task Sync Node



مقدمة وفكرة المشروع:

في ظل التطور المتسارع لبيئات العمل البرمجية والتحول العالمي نحو العمل الهجين والموزع، برزت الحاجة الماسة إلى أدوات تقنية متقدمة تضمن استمرارية تدفق العمل (Workflow) بكفاءة عالية ودون انقطاع. يبرز تحدي تزامن المهام كواحد من أهم التحديات التي تواجه الفرق التقنية، حيث يتطلب الأمر وجود وسيط تقني موثوق يضمن أن جميع أعضاء الفريق يشاهدون حالة المشروع الحقيقية في الوقت الفعلي، مما يمنع حدوث تعارض في البيانات (Data Conflicts) أو تكرار الجهود المبذولة.

يأتي نظام Task Server Pro كحل جذري لهذه التحديات، وهو منصة توزيع مهام متقدمة مبنية على هيكلية الزبون-الخادم (Client-Server Architecture) باستخدام لغة البرمجة C. يعمل النظام كخادم مركزي (Centralized Server) عالي الأداء، مخصص لاستقبال ومعالجة طلبات إدارة العمليات من زبائن (Clients) متعددين وموزعين جغرافياً.

يعتمد النظام في جوهره على بروتوكولات الشبكة الموثوقة، وتحديدًا بروتوكول TCP، لضمان نقل الأوامر الحساسة (مثل تعيين مهمة لموظف أو تغيير المواعيد النهائية) بسرعة وأمان عاليين. ولتجاوز قصور الأنظمة التقليدية في معالجة الطلبات المتزامنة، يتبنى المشروع تقنيات المعالجة المتعددة (Multi-threading) عبر مكتبة pthread.h؛ مما يتيح للخادم خدمة عدة زبائن في آن واحد دون حدوث حالة حظر (Blocking) أو توقف للخدمة.

بالإضافة إلى ذلك، يضمن النظامديمومة البيانات (Persistence) من خلال الأرشفة الفورية في ملفات نصية، ويهدف إلى بناء بيئة عمل تعاونية متكاملة يتم إدارتها وتطويرها وفق معايير الصناعة البرمجية الحديثة عبر مستودعات GitHub. وبذلك، يمثل Task Server Pro حجر الزاوية لفهم كيفية بناء تطبيقات شبكية متينة تجمع بين كفاءة التعامل مع السوكات (Sockets) ودقة إدارة البيانات الموزعة.

الأهداف التقنية للمشروع:

يسعى المشروع إلى تحقيق حزمة من الأهداف التقنية التي تضمن بناء نظام شبكي متين وقابل للتوسع، وتتمثل هذه الأهداف فيما يلي:

1. بناء وإدارة قاعدة بيانات مركزية (Centralized Data Management):

لا يقتصر الهدف على مجرد تخزين البيانات، بل يمتد لبناء مستودع بيانات مركزي يعمل كمصدر وحيد للحقيقة (Single Source of Truth) لجميع الزبائن المتصلين.

هيكلية البيانات: تصميم سجلات برمجية (Structs) متطورة قادرة على استيعاب كافة تفاصيل المهمة (المعرف، العنوان، الحالة، والتوقيت).

ديمومة البيانات (Persistence): ضمان عدم فقدان المعلومات من خلال تقنيات الكتابة الفورية والمزامنة بين الذاكرة العشوائية (RAM) والملفات النصية الدائمة (tasks.txt)، مما يحمي النظام من ضياع البيانات في حالات الانقطاع المفاجئ.

2. الإدارة الكاملة لدورة حياة المهمة (Full Task Lifecycle Management):

يهدف النظام إلى أتمتة العمليات الأساسية للمهام من لحظة الإنشاء وحتى الأرشفة، وذلك من خلال تطبيق عمليات CRUD المتقدمة:

التحكم الديناميكي: توفير إمكانية الانتقال السلس للمهمة بين الحالات المختلفة (مثل: قيد الانتظار، قيد التنفيذ، مكتملة).

بروتوكول الأوامر (Op-Codes): ابتكار نظام ترميز رقمي (مثل الرموز ١٠١، ٣٠٣) لتنفيذ العمليات بسرعة فائقة وتقليل حجم البيانات المتبادلة عبر الشبكة، مما يحسن من زمن الاستجابة (Latency).

3. التزامن الفوري ومعالجة الاتصالات المتعددة (Real-time Sync & Concurrency):

يعتبر هذا الهدف هو التحدي التقني الأكبر في الأنظمة الموزعة، ويهدف المشروع إلى حله عبر:

المعالجة المتعددة (Multi-threading): استخدام مكتبة pthread.h لتمكين الخادم من خدمة عشرات الزبائن في آن واحد، حيث يتم تخصيص "خيط معالجة" مستقل لكل اتصال، مما يضمن استمرارية الخدمة لجميع المستخدمين دون تأخير.

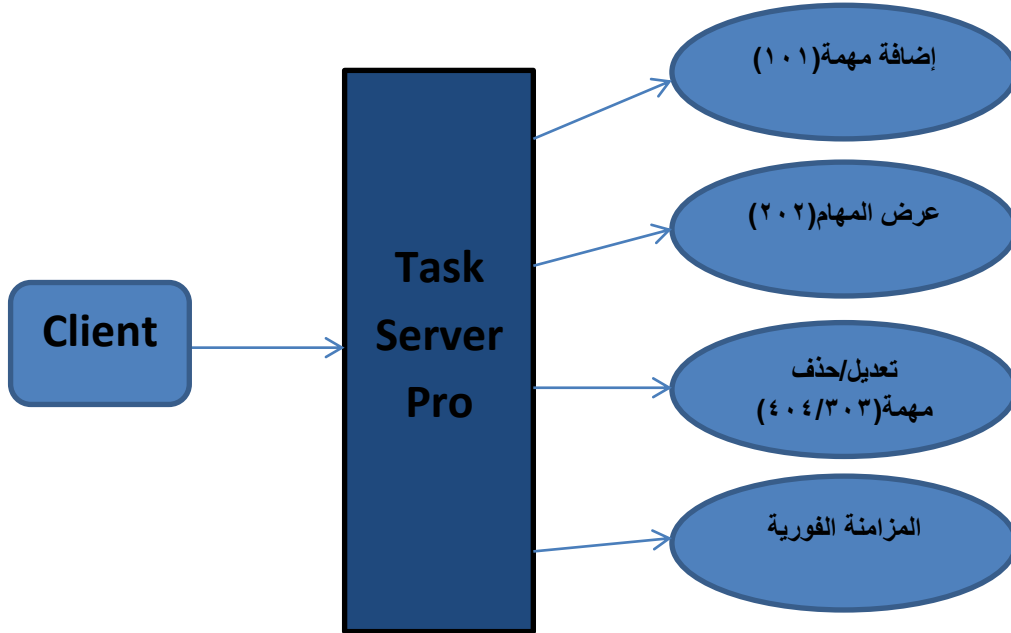
إدارة الوصول المشترك (Concurrency Control): تطبيق تقنيات المزايج (Mutex Locks) لمنع حدوث حالة السباق (Race Condition)، وضمان أن عمليات التعديل على المهام تتم بترتيب منطقي وسليم يمنع تضارب البيانات عند محاولة أكثر من مستخدم تعديل نفس المهمة في اللحظة ذاتها.

4. موثوقية الاتصال الشبكي (Network Reliability):

الاعتماد الكلي على بروتوكول TCP لضمان وصول كافة الحزم البيانية (Packets) بالترتيب الصحيح ودون نقصان، وهو أمر حيوي جداً في تطبيقات إدارة المهام حيث لا يمكن التغاضي عن فقدان أي أمر أو تعديل بسيط.

تحليل حالات الاستخدام

يوضح المخطط التالي (Use Case Diagram) طبيعة الأدوار والتفاعلات الديناميكية بين الزبون كفاعل أساسي للنظام وبين الخادم المركزي، حيث يبرز العمليات الجوهرية التي تتم عبر الشبكة لضمان إدارة وتزامن فعال للمهام و لدينا المخطط يوضح تنفيذ المهام:



تعد حالات الاستخدام حجر الزاوية في فهم التفاعلات الديناميكية بين "الزبون" و"الخادم". يركز نظام Task Server Pro على نموذج "الطلب-الاستجابة" لضمان مزامنة البيانات بين المستخدمين الموزعين والخادم المركزي، وفيما يلي توصيف موسع لهذه الحالات:

1. حالة استخدام: إضافة مهمة جديدة (Add New Task):

الوصف: تتيح هذه الحالة للمستخدم إدخال بيانات مهمة جديدة ليتم تخزينها مركزياً ومشاركتها مع الفريق.

التفاعل التقني: تبدأ العملية بإرسال الزبون لرمز العملية الموحد ١٠١. يتبع ذلك تغليف بيانات المهمة ضمن هيكل البيانات Task Struct (المتضمن للعنوان والحالة) وإرساله عبر السوكت.

استجابة السيرفر: عند استلام الرمز ١٠١، يقوم السيرفر بالتحقق من سلامة البيانات (Validation)، ثم يولد معرفاً فريداً (Unique ID) للمهمة. بعد ذلك، يتم حجز قفل المزلاج (Mutex Lock) لضمان الكتابة الآمنة في المصفوفة المركزية وفي ملف التخزين الدائم tasks.txt.

النتيجة: يرسل السيرفر إشارة تأكيد ١٠٠ OK للزبون، وتظهر المهمة فوراً في قائمة المهام العامة.

2. حالة استخدام: استعراض وتحديث قائمة المهام (List & Sync Tasks):

الوصف: تمكن المستخدم من استرجاع كافة المهام الحالية من الخادم لضمان العمل على أحدث نسخة من البيانات.

التفاعل التقني: يرسل الزبون رمز الاستعلام ٢٠٢ عبر قناة اتصال TCP الموثوقة.

استجابة السيرفر: يقوم الخادم بالوصول إلى مصفوفة المهام في الذاكرة العشوائية (RAM)، ويقوم بمسح كافة السجلات الحالية، ثم يرسلها بشكل متسلسل إلى الزبون.

النتيجة: يتم تحديث واجهة السطر البرمجي (CLI) لدى الزبون لتعكس الحالة الراهنة للمشروع، مما يمنع العمل على مهام قديمة أو ملغاة.

3. حالة استخدام: تعديل حالة المهمة أو حذفها (Update/Delete Task):

الوصف: تتيح إدارة دورة حياة المهمة، سواء بتغيير حالتها (مثلاً: من قيد التنفيذ إلى مكتملة) أو حذفها نهائياً عند الضرورة.

التفاعل التقني: يستخدم الزبون الرمز ٣٠٣ لطلب التعديل أو الرمز ٤٠٤ لطلب الحذف، مع إرفاق المعرف الرقمي (ID) للمهمة المستهدفة.

استجابة السيرفر: يقوم السيرفر بالبحث عن المعرف المطلوب؛ في حال التعديل، يتم تحديث الحقول المطلوبة، وفي حال الحذف، يتم إزاحة السجلات في المصفوفة وإعادة ترتيبها. وفي كلتا الحالتين، يتم تحديث الملف النصي فوراً لضمان ديمومة البيانات (Persistence).

النتيجة: استجابة فورية للزبون بنجاح العملية، مع تحديث تلقائي لقاعدة البيانات المركزية.

4. حالة استخدام: المزامنة الجماعية (Global System Sync):

الوصف: ضمان أن جميع الزبائن المتصلين في نفس اللحظة يتلقون التحديثات فور وقوعها.

التفاعل التقني: تعتمد هذه الحالة على تقنية البث (Broadcasting) أو إرسال تنبيهات عبر خيوط المعالجة (Threads) النشطة.

استجابة السيرفر: بمجرد نجاح أي عملية إضافة أو تعديل، يقوم السيرفر بإخطار جميع خيوط المعالجة المتصلة حالياً بوجود تحديث جديد.

النتيجة: تحديث فوري لكافة واجهات المستخدمين المتصلين، مما يقدم حلاً هندسياً لمشكلة ضياع التزامن ويحقق مبدأ العمل الجماعي الحقيقي.



المتطلبات التقنية للنظام:

تعتمد فعالية نظام **Task Server Pro** على تكامل قوي بين جهة الخادم وجهة الزبون، مع توفير بنية تحتية برمجية تدعم المزامنة والأداء العالي:

أولاً: متطلبات الخادم (Server-Side Requirements):

يعمل الخادم كعقل مدبر للنظام، ويجب أن يتمتع بالقدرات التالية:

دعم المعالجة المتعددة (Multithreading): الاعتماد الكلي على مكتبة **pthread.h**

لتمكين الخادم من إنشاء خيوط معالجة (**Threads**) مستقلة لكل اتصال جديد. هذا يضمن أن النظام لا يتوقف (**Non-blocking**) عند معالجة طلبات أحد الزبائن، مما يسمح بخدمة عدة مستخدمين في آن واحد.

إدارة التزامن (Concurrency Control): استخدام المزاليج (**Mutex Locks**) كأداة برمجية أساسية لحماية مصفوفة المهام المخزنة في الذاكرة (**RAM**) من الوصول المتزامن، مما يضمن سلامة البيانات عند قيام أكثر من زبون بعملية تعديل في نفس اللحظة.

بيئة التشغيل الموثوقة: يفضل تشغيل الخادم على بيئة **Linux** (مثل **Ubuntu**) لضمان الاستقرار التام في التعامل مع السوكاتات (**Sockets**) وإدارة الذاكرة.

ثانياً: متطلبات الزبون (Client-Side Requirements):

تم تصميم جهة الزبون لتكون خفيفة الوزن وسهلة الاستخدام، مع التركيز على:

واجهة السطر البرمجي (CLI): بناء واجهة تفاعلية بسيطة تعتمد على إدخال الأوامر النصية، مما يقلل من استهلاك موارد الجهاز ويسمح للمطورين بإدارة المهام بسرعة من خلال الأوامر (**add, list, update, delete**).
بروتوكول المزامنة والطلبات: تزويد الزبون بوحدة برمجية قادرة على تغليف البيانات وإرسال طلبات المزامنة (**Sync Requests**) بشكل دوري أو يدوي، لضمان توافق النسخة المحلية للمهام مع ما هو موجود في السيرفر المركزي.
إدارة الاتصال الشبكي: القدرة على معالجة انقطاع الاتصال المفاجئ وإعادة المحاولة لضمان عدم ضياع الأوامر المرسلة.

ثالثاً: بيئة التطوير والربط (Development & Integration):

لغة البرمجة: استخدام لغة **C** القياسية لضمان الوصول المباشر إلى موارد النظام والتحكم الكامل في السوكاتات.

أدوات البناء والمراقبة: استخدام مترجم **GCC** وإدارة الكود عبر مستودع **GitHub** المخصص لضمان تتبع النسخ (**Version Control**) والتنسيق بين الأعضاء.

بروتوكول التطبيق واتفاقية التواصل:

يعتبر بروتوكول التطبيق هو اللغة المشتركة التي تتيح للزبون وال خادم فهم الطلبات والاستجابات بشكل موحد. يعتمد النظام على نظام ترميز رقمي يُعرف ب رموز العمليات (Op-Codes) ، والتي تهدف إلى تقليل حجم البيانات المتبادلة وتسريع عملية المعالجة عبر الشبكة.

أولاً: شرح رموز العمليات والاستجابة:

تم تخصيص رموز فريدة لكل إجراء وظيفي لضمان دقة التنفيذ وتجنب الغموض في الأوامر ، كما هو موضح في الجدول التالي:

رمز العملية (Code)	نوع الإجراء (Action Type)	الوصف التقني (Technical Description)
101	Create Task	طلب إضافة مهمة جديدة وتجهيز السيرفر لاستقبال بيانات الهيكل (Struct).
202	List/Read Tasks	طلب استرجاع ومزامنة كافة المهام الموجودة في قاعدة البيانات المركزية.
303	Update Task	طلب تعديل حالة أو تفاصيل مهمة موجودة بناءً على المعرف الخاص بها (ID).
404	Delete Task	طلب حذف مهمة نهائياً من سجلات النظام المركزية.
100	Response: OK	رسالة تأكيد من السيرفر بنجاح العملية وتحديث ملف التخزين بنجاح.
500	Response: Error	رسالة تشير إلى فشل العملية (مثل عدم وجود المعرف أو خطأ في الوصول المشترك).

ثانياً: كيفية تغليف البيانات (Data Encapsulation):

تغليف البيانات هو عملية تحويل المعلومات إلى حزم برمجية مهيأة للنقل الآمن عبر السوكت

(Socket):

بنية الحزمة (Packet Structure): يتم إرسال البيانات ككتلة واحدة (Buffer) تبدأ ب رأس الحزمة (Header) الذي يحتوي على الرمز الرقمي، يليه جسم الحزمة (Payload) الذي يحتوي على تفاصيل المهمة (العنوان، الحالة، الموعد).

التسلسل (Serialization): بما أننا نستخدم لغة C ، يتم تحويل سجل البيانات Task Struct إلى سلسلة من البايتات (Bytes) ليتمكن بروتوكول TCP من نقلها كتدفق بياني (Byte Stream) دون فقدان الترتيب أو المحتوى.

فك التغليف (De-encapsulation): عند وصول البيانات للسيرفر، يتم قراءة الرمز الرقمي أولاً لتحديد نوع العملية، ثم يتم تفريغ بقية البيانات مجدداً في هيكل البيانات (Struct) داخل الذاكرة لاتخاذ الإجراء المناسب.

الخاتمة:

يمثل مشروع **Task Server Pro** نموذجاً تطبيقياً متكاملًا لنظم إدارة المهام الموزعة القائمة على هيكلية (Client-Server). من خلال هذا العمل، تم النجاح في بناء منصة مركزية قادرة على معالجة التحديات الجوهرية في تطبيقات الشبكات، بدءاً من ضمان موثوقية نقل البيانات باستخدام بروتوكول **TCP**، وصولاً إلى تحقيق مبدأ التزامن الفوري بين زبائن متعددين عبر تقنيات **Multi-threading**.

القيمة التقنية والمكتسبات:

لم يكن الهدف من المشروع مجرد بناء أداة برمجية لإدارة المهام، بل كان تحدياً هندسياً في كيفية إدارة الموارد المشتركة. لقد مكنا استخدام لغة **C** من فهم أعمق لكيفية التعامل المباشر مع الذاكرة و السوكاتات (**Sockets**)، كما أن تطبيق مفاهيم **Mutex Locks** ضمن مكتبة **pthread.h** قد وفر حلاً عملياً لمنع تعارض البيانات وضمان سلامتها (**Data Integrity**) في بيئة عمل عالية التزامن.

الدروس المستفادة:

أثبتت مراحل تطوير النظام أهمية التخطيط المسبق لبروتوكول التطبيق (**Application Protocol**) ؛ حيث ساهم تصميم رموز العمليات (**Op-Codes**) في تبسيط عملية التواصل وتقليل حجم الحزم المتبادلة. كما عزز العمل الجماعي عبر منصة **GitHub** من مهارات الفريق في إدارة النسخ المصدري وحل النزاعات البرمجية بشكل احترافي، مما يضمن ديمومة المشروع وقابلية صيانتها مستقبلاً.

التطلعات المستقبلية (Future Work):

نطمح في المراحل المتقدمة إلى توسيع نطاق **Task Server Pro** ليشمل:
الأمن والتشفير: دمج بروتوكولات التشفير (مثل **SSL/TLS**) لتأمين البيانات المتبادلة بين الزبائن والخادم.
واجهات متقدمة: تطوير واجهات رسومية (**GUI**) لزيادة سهولة الاستخدام، مع الحفاظ على متانة السيرفر المركزي.
التوسع السحابي: تهيئة النظام للعمل كخدمة سحابية (**SaaS**) تدعم آلاف الاتصالات المتزامنة باستخدام تقنيات موازنة الحمل (**Load Balancing**).

رابط مستودع GitHub:

<https://github.com/wwwhasyk678-collab/Task-Server-Pro.git>